Vision 기본 모델 Day 2

CNN 대표 구조 — LeNet · VGG · ResNet

HYUNDAI AI Insight Campus · 온디바이스 AI · Vision 기본 모델 2/5

2026. 10. 28 (수) 09:00–18:00 · 8교시 · Google Colab T4 GPU

CIFAR-10 | Conv2d | LeNet-5 | VGG | BatchNorm | ResNet-20

강사 : 김정인 · 김구수

오늘의 학습 목표

수업이 끝나면 여러분은 다음을 할 수 있습니다

1 CIFAR-10 · MLP 의 한계

32×32 RGB 10 클래스를 정규화해 불러오고 , MLP (1,707,274 파라미터) 가 펼치기로 공간 정보를 잃는 이유를 설명한다

2 Conv 계산

출력 크기 ⌊(N + 2P − K)/S⌋ + 1 · 파라미터 Cout·(Cin/G)·K² · 수용 영역을 손으로 계산한다

3 LeNet · AlexNet

LeNet-5 (62,006 — MLP 의 1/27) 를 구현해 MLP 와 비교하고 , AlexNet 이 연 ImageNet 시대를 정리한다

4 VGG · BatchNorm

3×3 쌓기로 VGG-mini (307,946) 를 만들고 , 깊은 plain 망의 기울기 소실을 BatchNorm 으로 줄인다

5 ResNet

y = ReLU(F(x) + x) BasicBlock 으로 ResNet-20 (272,474) 을 만들고 지름길의 효과를 확인한다

6 미니 프로젝트

증강 · OneCycleLR 로 ResNet-20 을 20 epoch 학습해 test 0.85 이상 (CIFAR-10) 에 도전한다

시간표

8 교시 × 50 분 · 점심 12:00–13:00 · 오전 MLP → Conv → LeNet, 오후 VGG → BN → ResNet → 미니 프로젝트

교시	시간	주제	형태	애니메이션
1	09:00–09:50	Day 1 복습 퀴즈 · CIFAR-10 과 MLP 의 한계	퀴즈 + 강의	D2-1 MLP 의 한계
2	10:00–10:50	Conv 다시 보기 — 출력 크기 · 파라미터 · 수용 영역	강의 + 실습	D2-2 Conv2d 의 인자
3	11:00–11:50	LeNet-5 · AlexNet	강의 + 실습	D2-3 LeNet-5 의 shape 흐름
4	13:00–13:50	VGG — 3×3 쌓기	강의 + 실습	D2-4 3×3 쌓기와 수용 영역
5	14:00–14:50	깊이의 문제 · BatchNorm	강의 + 실습	D2-5 기울기 소실과 BatchNorm
6	15:00–15:50	ResNet — Residual 지름길	강의 + 실습	D2-6 Residual 지름길
7	16:00–16:50	학습 기법 — 증강 · 학습률 스케줄 · MAC	강의 + 실습	D2-7 데이터 증강 · 학습률 스케줄
8	17:00–17:50	미니 프로젝트 — CIFAR-10 ResNet-20 85% 도전	개인 실습	D2-8 ResNet-20 학습 · 오답 분석

오늘의 자료 · 온라인 링크

예측 (애니메이션) → 검증 (노트북) → 점검 (워크시트) — 모두 브라우저에서 바로 열립니다

기본 주소 (Day 2)

https://imguru-mooc.github.io/AI_CAMPUS/VB-day2/

animation.html	애니메이션 8 개 (D2-1 ~ D2-8) — #d2-6 처럼 바로가기
worksheet.html	개념 워크시트 — #t1 ~ #t8 교시 바로가기
VB_Day2_lecture.ipynb	강의 노트북 — 1~8 교시 문제 + 미니 프로젝트 (Colab · T4 GPU)
VB_Day2.pdf	오늘 슬라이드 PDF
data/resnet20_cifar10.pt	강사가 학습한 ResNet-20 가중치 — CPU 런타임일 때 8 교시 이어서 진행
Day 1 복습 퀴즈	09:00 시작 — 강사가 Google Forms 링크 공유 (20 문항 · 15 분)

노트북은 " 파일 → Drive 에 사본 저장 " 후 작업 · 런타임 → 런타임 유형 변경 → T4 GPU · 하드웨어 없이 데이터만으로 진행합니다

오늘의 큰 그림 — MLP 에서 ResNet 까지

같은 CIFAR-10 에 네 모델을 차례로 만듭니다 . 파라미터와 연산량 (MAC) 을 함께 보세요

MLP

params

1,707,274

MAC 1,706,496

펼치기 → 이웃 관계 손실

→

LeNet-5 1998

params

62,006

MAC 651,720

Conv · Pool 2 단 · MLP 의 1/27

→

VGG-mini 2014

params

307,946

MAC 38,653,952

3×3 블록 반복 + BN

→

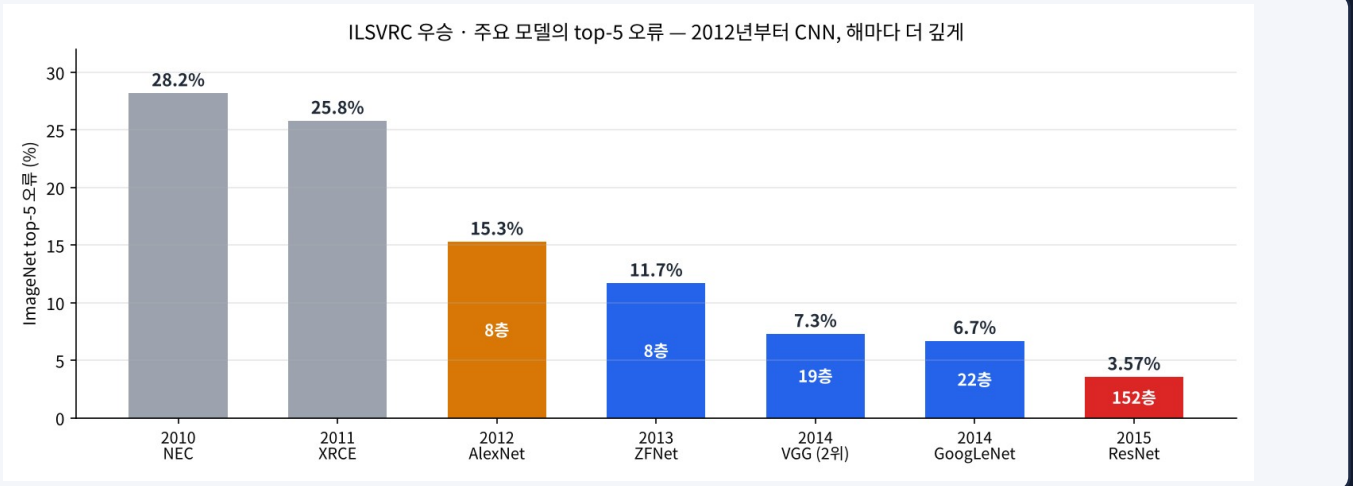
ResNet-20 2015

params

272,474

MAC 40,813,184

지름길 y = ReLU(F(x) + x)



오늘의 데이터 — CIFAR-10



32×32 RGB
= 3072 값
(MNIST 784)

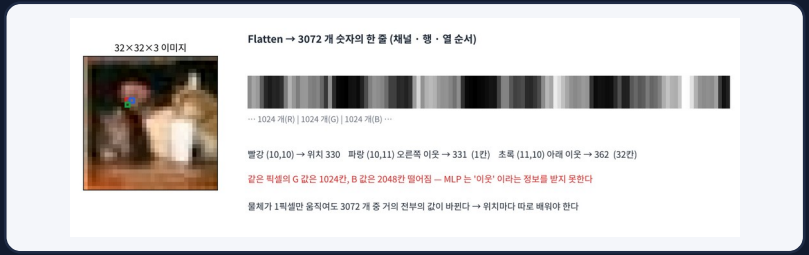
10 클래스
train 50,000
test 10,000

1 Day 1 복습 퀴즈 · CIFAR-10 과 MLP 의 한계

09:00–09:50

학습 목표

- 09:00–09:15 Vision Day 1 복습 퀴즈 (20 문항)
- CIFAR-10: 32×32 RGB · 10 클래스
- 채널별 정규화 — train 으로만 계산
- MLP 는 펼치면서 이웃 관계를 잃는다



노트북 실습

1. 문제 1-1 채널별 mean · std
2. 문제 1-2 MLP 3072-512-256-10
3. 문제 1-3 MLP · Adam 3 epoch

▶ 애니메이션 D2-1 MLP 의 한계

핵심 개념

CIFAR-10

```
airplane automobile bird cat deer dog frog horse ship truck
train 50,000 (클래스당 5,000) · test 10,000 (클래스당 1,000)
```

정규화

```
data01 = train_set.data / 255.0   # (50000, 32, 32, 3) HWC
MEAN = data01.mean(axis=(0, 1, 2)) → (0.4914, 0.4822, 0.4465)
STD  = data01.std(axis=(0, 1, 2))  → (0.2470, 0.2435, 0.2616)
```

로더

```
DataLoader(train_set, batch_size=128, shuffle=True,
           num_workers=2, pin_memory=True)   → 391 step/epoch
```

MLP

```
3072·512+512 = 1,573,376 (92%) · 131,328 · 2,570
합계 1,707,274   3 epoch test 0.4994 (실행 예)
```

체크포인트

펼치면 바로 아래 픽셀은 32 칸, 같은 위치의 다른 채널은 1024 칸 떨어진다 — 이웃 관계와 이동 불변성이 사라지고, 파라미터의 92% 가 첫 층에 몰린다.

2 Conv 다시 보기 — 출력 크기 · 파라미터 · 수용 영역

10:00–10:50

학습 목표

- nn.Conv2d(in, out, kernel_size, stride, padding, groups, bias)
- O = ⌊(N + 2P − K)/S⌋ + 1
- 파라미터 = Cout·(Cin/G)·K² + Cout
- 수용 영역 rf ← rf + (K−1)·jump, jump ← jump·S

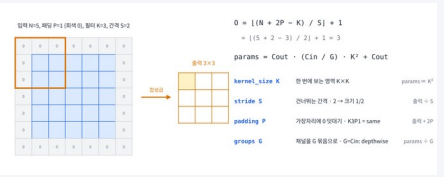
노트북 실습

1. 문제 2-1 출력 크기 함수
2. 문제 2-2 파라미터 수 함수
3. 문제 2-3 수용 영역 함수
4. 문제 2-4 크기 절반 · 채널 2 배 설계

▶ 애니메이션 D2-2 Conv2d 의 인자

핵심 개념

출력 크기



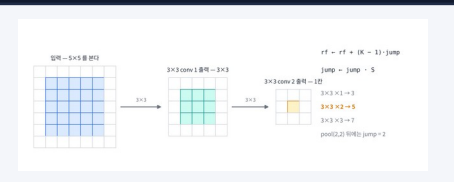
2-1 (N,K,S,P)

```
(32,5,1,0)→28   (32,3,1,1)→32 same   (32,3,2,1)→16
(28,2,2,0)→14   (224,7,2,3)→112 ResNet 첫 층   (7,3,2,0)→3
```

2-2

```
Conv2d(3→6, 5×5) 456          Conv2d(64→128, 3×3) 73,856
(64→64, 3×3, groups=64) 640 =  일반 36,928 의 1/57.7
(64→128, 1×1, bias=False) 8,192   (16→32, 3×3, bias=False) 4,608
```

수용 영역



체크포인트

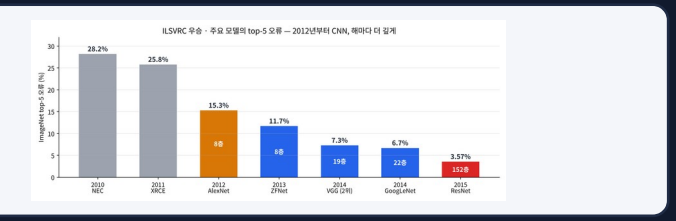
2-4: (N,64,32,32) → (N,128,16,16) 은 nn.Conv2d(64, 128, 3, stride=2, padding=1). 크기 1/2 → 위치 1/4, 채널 2 배 → 연산량 절반 . BN 앞 Conv 는 bias=False.

3 LeNet-5 · AlexNet

11:00–11:50

학습 목표

- LeNet-5 (1998, Yann LeCun) — Conv · Pool 2 단 + FC 3 층
- 오늘 판 : 3 채널 · ReLU · MaxPool (원래 흑백 · tanh · 평균 풀링)
- AlexNet (2012) — ReLU · Dropout · 증강 · GPU



노트북 실습

1. 문제 3-1 LeNet-5 nn.Module (62,006)
2. 문제 3-2 LeNet-5 vs MLP 3 epoch
3. 문제 3-3 AlexNet 파라미터 세기

▶ 애니메이션 D2-3 LeNet-5 의 shape 흐름

핵심 개념

shape 흐름



파라미터

```
456 + 2,416 + 48,120 + 10,164 + 850 = 62,006
MAC conv1 352,800 + conv2 240,000 + FC 58,920 = 651,720
```

3-2 (실행 예)

```
MLP        1,707,274 params    3 epoch test 0.4994
LeNet-5    62,006 params       3 epoch test 0.5489
```

AlexNet

```
ILSVRC 2012 top-5 오류 15.3% (2 위 26.2%) · 8 층 · GTX 580 ×2
ImageNet 1,000 클래스 · 약 128 만 장 · 224×224
전체 61,100,840 · classifier.1.weight 37,748,736 (61.8%)
```

체크포인트

파라미터는 MLP 의 1/27 — 이웃 픽셀을 함께 보고 같은 필터를 모든 위치에서 다시 쓰는 귀납적 편향 (inductive bias). AlexNet 은 합성곱 5 층이 약 247 만 (4%), 나머지는 거의 FC.

4 VGG — 3×3 쌓기

13:00–13:50

학습 목표

- VGG (2014, Oxford Visual Geometry Group) — 모든 합성곱 3×3, 16~19 층
- 3×3 두 층 = 5×5 수용 영역, 파라미터 28% ↓
- 3×3 세 층 = 7×7, 45% ↓ · ReLU 도 더 많이
- 블록 = [Conv → BN → ReLU] × n → MaxPool

노트북 실습

1. 문제 4-1 5×5 vs 3×3×2 (C=64)
2. 문제 4-2 vgg_block · VGG-mini
3. 문제 4-3 VGG-mini 3 epoch

▶ 애니메이션 D2-4 3×3 쌓기와 수용 영역

핵심 개념

3×3 쌓기



4-1 (C=64)

```
5×5 102,400   vs   3×3×2  73,728
7×7 200,704   vs   3×3×3 110,592     (bias 없음)
```

vgg_block

```
[nn.Conv2d(cin, cout, 3, padding=1, bias=False),
 nn.BatchNorm2d(cout), nn.ReLU()] × n_convs  →  nn.MaxPool2d(2)
```

VGG-mini

```
vgg_block(3,32,2) (N,32,16,16) → vgg_block(32,64,2) (N,64,8,8)
→ vgg_block(64,128,2) (N,128,4,4) → Flatten → Linear(2048,10)
307,946 params · MAC 38,653,952 · 3 epoch test 0.7526 (실행 예)
```

체크포인트

원래 VGG 에는 BN 이 없다 (BN 은 2015). 파라미터는 conv 286,560 + BN 896 + Linear 20,490. LeNet 대비 파라미터 약 5 배, MAC 약 59 배 — padding=1 로 해상도를 유지하며 채널을 늘리기 때문.

5 깊이의 문제 · BatchNorm

14:00–14:50

학습 목표

- plain-N: 3×3 Conv(16ch) N−1 개 + GAP + Linear (plain-8 14,538 · plain-20 42,378)
- 깊을수록 입력 쪽 기울기가 작아진다 — 기울기 소실
- BN: 채널마다 (배치 · 세로 · 가로) 통계로 정규화 → γ, β
- 추론은 running 평균 · 분산 — model.eval()

노트북 실습

1. 문제 5-1 층별 기울기 노름
2. 문제 5-2 plain-8 · 20 ± BN 학습
3. 문제 5-3 BN 파라미터 · buffer

▶ 애니메이션 D2-5 기울기 소실과 BatchNorm

핵심 개념

5-1 기울기 (실행 예)

```
conv          first (input)   last
plain-8       3.2e-04         2.8e-02
plain-20      1.1e-09         2.2e-02
plain-20+BN   3.4e-01         1.7e-01
```

5-2 (실행 예)

```
10,000 장 · SGD 0.05 momentum 0.9 · 3 epoch · 마지막 epoch
              train loss   test
plain-8       2.304        0.1000
plain-8+BN    1.699        0.3037
plain-20      2.303        0.1000
plain-20+BN   1.785        0.2696
```

BN

$$\hat{x} = (x - \mu_{batch}) / \sqrt{(\sigma^2_{batch} + \varepsilon)}, \quad y = \gamma \cdot \hat{x} + \beta$$

BatchNorm2d(16): 학습 파라미터 32 (γ 16 + β 16)
buffer: running_mean · running_var · num_batches_tracked

체크포인트

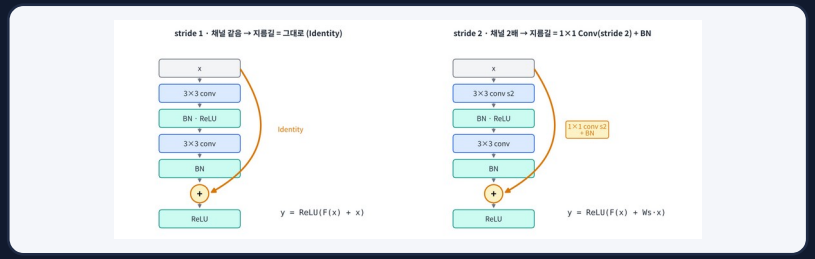
실행 예 : BN 없으면 8 층 ·20 층 모두 손실 ≈ 2.30 (ln 10) — 학습 실패 . BN 을 넣으면 둘 다 배우지만 20 층이 8 층보다 train 손실도 높다 → degradation → 6 교시 ResNet. 흔한 실수 : eval() 누락 · 배치 크기 1 · BN 앞 conv bias=True.

6 ResNet — Residual 지름길

15:00–15:50

학습 목표

- BN 이 있어도 56 층 plain 이 20 층보다 train 오류 ↑ (degradation)
- y = ReLU(F(x) + x) — 항등은 F = 0 만 배우면 끝
- ∂y/∂x = F′(x) + 1 — 기울기가 지름길로 흐른다



노트북 실습

1. 문제 6-1 BasicBlock
2. 문제 6-2 ResNet (make_layer) · ResNet-20
3. 문제 6-3 plain-20 vs ResNet-20

▶ 애니메이션 D2-6 Residual 지름길

핵심 개념

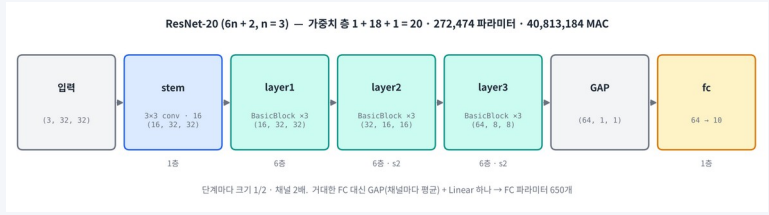
BasicBlock

```
conv1 3×3(stride) → BN → ReLU → conv2 3×3 → BN → (+ shortcut) → ReLU
shortcut = 1×1 Conv(stride, bias=False) + BN   또는   nn.Identity()
```

6-1

```
BasicBlock(16, 32, stride=2): (2,16,32,32) → (2,32,16,16) · 14,528
= conv1 4,608 + bn1 64 + conv2 9,216 + bn2 64 + 1×1 512 + BN 64
```

ResNet-20



6-3 (실행 예)

	test	loss	stem 기울기
plain-20	0.3803	1.641	9.8e-01
ResNet-20	0.4432	1.438	8.3e-01

체크포인트

ResNet-20 (6n+2, n=3): 272,474 params · MAC 40,813,184 — MLP 대비 파라미터 1/6, 연산 약 24 배 . GAP 덕분에 FC 는 650 개뿐 . 6-3 은 10,000 장 · SGD 0.05 · 3 epoch.

7 학습 기법 — 증강 · 학습률 스케줄 · MAC

16:00–16:50

학습 목표

- 증강은 train 에만 — 정답이 바뀌지 않는 변형만
- OneCycleLR: 0.004 → 30% 지점 0.1 → 4e-7
- scheduler.step() 은 배치마다
- 혼동 행렬 · forward hook 으로 MAC 세기

노트북 실습

1. 문제 7-1 증강 transform
2. 문제 7-2 OneCycleLR 곡선
3. 문제 7-3 혼동 행렬
4. 문제 7-4 hook 으로 MAC 세기

▶ 애니메이션 D2-7 데이터 증강 · 학습률 스케줄

핵심 개념

증강

```
transforms.Compose([
    transforms.RandomCrop(32, padding=4),   # 최대 ±4 픽셀 이동
    transforms.RandomHorizontalFlip(),      # 50%
    transforms.ToTensor(), transforms.Normalize(MEAN, STD)])
```

OneCycleLR

```
OneCycleLR(opt, max_lr=0.1, epochs=15, steps_per_epoch=391)
0.004 → step 약 1,759 에서 최대 0.1 → 5,865 step 끝 4e-7
```

혼동 행렬

```
M = np.bincount(true * K + pred, minlength=K*K).reshape(K, K)
정확도 = 대각합 / 전체 · VGG-mini 최다 혼동 cat → dog (180장)
```

MAC

```
Conv  Cout·OH·OW·(Cin/G)·K²      Linear  in·out
MLP 1,706,496 · LeNet-5 651,720
VGG-mini 38,653,952 · ResNet-20 40,813,184
```

체크포인트

MNIST 에 좌우 뒤집기는 금지 (정답이 바뀌는 변형). 그림으로 볼 때는 × std + mean.
steps_per_epoch ≠ len(loader) 이면 "Tried to step ... times" 오류. hook 은 다 쓰면 remove().

8 미니 프로젝트 — CIFAR-10 ResNet-20 85% 도전

17:00–17:50

강의 노트북 8 교시 · ResNet-20 + 증강 + SGD + OneCycleLR

```
EPOCHS = 20
aug_loader = DataLoader(aug_set, batch_size=128, shuffle=True)  # 391
opt = torch.optim.SGD(model.parameters(), lr=0.1, momentum=0.9,
                      nesterov=True, weight_decay=5e-4)
sched = OneCycleLR(opt, max_lr=0.1, epochs=EPOCHS,
                   steps_per_epoch=len(aug_loader))
```

과제	내용	배점
1	증강 로더 · SGD(nesterov, wd 5e-4) · OneCycleLR	20
2	ResNet-20 20 epoch — test ≥ 0.85 (GPU 학습)	35
3	오답 분석 — 혼동 행렬 · 헷갈린 쌍 3 개 · 오답 16 장	15
4	모델 비교표 (파라미터 · MAC · test)	10
5	결과 해석 (마크다운)	10 + 10

채점 셀 실행 → 자동 채점 80 점 + 강사 20 점 (과제 5 의 10 점)

칸 번호 `k` → 정답 `k // 10`, 예측 `k % 10`

▶ D2-8 ResNet-20 학습 · 오답 분석 — 먼저 보고 코드로 확인

목표 test ≥ 0.85 (Colab T4 약 7 분, 예상)

강사 사전 실행 20 epoch test 0.8587 · CPU 런타임이면 강사 가중치 (과제 2 제외)

모델 비교 (CIFAR-10, 이미지 한 장 기준)

모델	params	MAC	test (실행 예)
MLP	1,707,274	1,706,496	0.4994
LeNet-5	62,006	651,720	0.5489
VGG-mini	307,946	38,653,952	0.7526
ResNet-20	**272,474**	**40,813,184**	**목표 ≥ 0.85**

MLP · LeNet-5 · VGG-mini = Adam 3 epoch, ResNet-20 = 8 교시 20 epoch

흔한 혼동 cat ↔ dog · automobile ↔ truck · bird ↔ airplane

공정한 비교 같은 학습 조건 — 구조 × 학습법 2×2

온디바이스 메모리 = params × 4 bytes · 속도 ≈ MAC (지연시간 실측은 Day 5)

참고 원 논문 ResNet-20 (약 160 epoch) 91.25%

오늘의 핵심 정리

각 항목 옆 번호는 다시 볼 애니메이션

CIFAR-10 · MLP	32×32×3 = 3072 · MLP 1,707,274 (92% 가 첫 층) · 펼치면 이웃 관계 손실	D2-1
Conv2d	$O = \lfloor (N + 2P - K)/S \rfloor + 1$ · $C_{out} \cdot (C_{in}/G) \cdot K^2$ · $rf \leftarrow rf + (K-1) \cdot jump$	D2-2
LeNet · AlexNet	LeNet-5 62,006 (MLP 의 1/27) · AlexNet 2012 top-5 15.3% → ImageNet 시대	D2-3
VGG	3×3 두 층 = 5×5 수용 영역 (−28%) · 블록 반복 · VGG-mini 307,946	D2-4
BatchNorm	깊은 plain 망은 기울기 소실 · BN = 채널별 정규화 + γ, β · 추론은 eval()	D2-5
ResNet	$y = \mathrm{ReLU}(F(x) + x)$ · $\partial y/\partial x = F' + 1$ · ResNet-20 272,474 / MAC 40,813,184	D2-6
학습 기법	RandomCrop · Flip (train 만) · OneCycleLR 0.004 → 0.1 → 4e-7, 배치마다 step	D2-7
미니 프로젝트	ResNet-20 20 epoch — 목표 test ≥ 0.85 · 오답 분석 (cat ↔ dog)	D2-8

Day 3 로 가는 다리 — 경량 CNN · 전이학습 · Grad-CAM

오늘 만든 ResNet-20 · fit · count_macs 를 들고, 작은 장치와 실제 현장 데이터로 갑니다

Day 2 (오늘)		Day 3 (10/29 목)
`ResNet-20 · MAC 40,813,184`	→	ResNet-50 을 작은 장치에서? — 연산량 문제
`nn.Conv2d(groups=1)`	→	Depthwise Separable Conv — depthwise = groups
`VGG · ResNet 블록 쌓기`	→	MobileNetV2 · V3
`CIFAR-10 처음부터 학습`	→	전이학습 — 현장 안전 분류기
`혼동 행렬 · 오답 그림`	→	Grad-CAM — 모델이 어디를 보았나
`fit · count_macs · resnet20_cifar10.pt`	→	**그대로 사용**

연산량 → Depthwise Separable Conv → MobileNetV2 · V3 → 전이학습 → Grad-CAM

오늘 저장한 resnet20_cifar10.pt 와 fit · count_macs 를 Day 3 에서 다시 씁니다 — 노트북을 Drive 에 사본 저장해 두세요